## Analysis & Complexity (Steps) – Recursive Algorithm

**Steps Analysis**

1. The main program starts by reading the integer n, which represents the size of the square matrix.
2. Then the program reads the elements of the matrix matrix[n][n].
3. The recursive function solve(matrix, n, 0, 0, 0) is called with the initial index i = 0 and both primarySum and secondarySum initialized to 0.
4. In each recursive call, the algorithm processes one row of the matrix.
5. The element matrix[i][i] is added to primarySum (primary diagonal).
6. The element matrix[i][n − 1 − i] is added to secondarySum (secondary diagonal).
7. The function then calls itself with the next index i + 1 while passing the updated sums.
8. The recursion continues until the base case i == n is reached.
9. When the base case is reached, the algorithm returns the absolute difference between the two sums: |primarySum − secondarySum|.

**Time Complexity**

The recursive function is called once for each row of the matrix. Therefore, the number of recursive calls equals n.

Time Complexity: O(n)

**Space Complexity**

Since the algorithm uses recursion, each recursive call is stored in the call stack until the base case is reached.

Space Complexity: O(n)